

Requirements & Traceability

Functional requirements of the Cypress Real World App and the requirements traceability matrix (RTM)

Author	Tomas Xavier Santos
Project	QA Atelier (tom-xs/qa-atelier)
AUT	Cypress Real World App
Date	2026-08-02
Version	1.1

Contents

1. Introduction	3
1.1 Purpose	3
1.2 Application under test	3
1.3 Scope	3
2. Functional Requirements	3
2.1 Authentication (REQ-AUTH)	3
2.2 Users (REQ-USER)	4
2.3 Bank accounts (REQ-BANK)	4
2.4 Transactions (REQ-TX)	4
2.5 Notifications (REQ-NOTIF)	5
3. Requirements Traceability Matrix	5
4. Maintenance Rules	6

1. Introduction

1.1 Purpose

This document defines the functional requirements of the application under test (AUT) and maintains the requirements traceability matrix (RTM) linking them to test cases. It is the coverage contract of the QA Atelier project: what must be tested, and the proof of what is tested.

1.2 Application under test

The **Cypress Real World App (RWA)** is an open-source payment application built by Cypress as a realistic testing target — a Venmo-style service where users sign up, link bank accounts, send payments, request money, browse transaction feeds, like and comment, and receive notifications. It runs locally with a React frontend (port 3000) and an Express REST API (port 3001) over a file-based JSON database with a seed dataset.

Two verified characteristics shape the requirements and their tests:

- **Session-cookie authentication** — `POST /login` returns the user object and sets a `connect.sid` cookie; there is no bearer token.
- **Ephemeral state** — every `yarn start` re-seeds the database from `data/database-seed.json`; a restart is a factory reset.

1.3 Scope

In scope: auth, users, bank accounts, transactions, feeds, and notifications at API and UI level. **Out of scope:** performance/load, security penetration testing, iOS, and the optional OAuth providers (Auth0, Okta, Cognito, Google) — only the local username/password strategy is tested.

2. Functional Requirements

Requirements are testable statements with stable IDs. Priorities: **P0** = critical path, **P1** = every test cycle, **P2** = normal.

2.1 Authentication (REQ-AUTH)

ID	Requirement	Priority
REQ-AUTH-001	A user with valid credentials can log in; the server returns the user object and establishes a session cookie	P0

REQ-AUTH-002	A user with an invalid password cannot log in; the API returns 401 and the UI shows an error message	P0
REQ-AUTH-003	A logged-in user can log out; the session is invalidated and protected pages redirect to sign-in	P1
REQ-AUTH-004	A new user can sign up with username, password, first and last name	P1
REQ-AUTH-005	An active session persists across page reloads (cookie-based); no re-login needed	P1

2.2 Users (REQ-USER)

ID	Requirement	Priority
REQ-USER-001	A logged-in user can search other users by name or username	P1
REQ-USER-002	A logged-in user can view another user's public profile (first name, last name, avatar)	P2
REQ-USER-003	A logged-in user can update their own account settings (name, email, phone)	P2

2.3 Bank accounts (REQ-BANK)

ID	Requirement	Priority
REQ-BANK-001	A logged-in user can link a bank account with bank name, routing number, account number	P1
REQ-BANK-002	A logged-in user can view their list of linked bank accounts	P1
REQ-BANK-003	A logged-in user can delete (soft-delete) a linked bank account	P2

2.4 Transactions (REQ-TX)

ID	Requirement	Priority
REQ-TX-001	A logged-in user with sufficient balance can pay another user an amount with a description; the transaction appears in feeds and balances update	P0

REQ-TX-002	A logged-in user can request money from another user; the request appears as pending for the recipient	P0
REQ-TX-003	Transaction feeds are filtered correctly: Mine, Friends, Public	P1
REQ-TX-004	The recipient of a money request can accept (creates payment) or reject it	P1
REQ-TX-005	A user can like and comment on a transaction	P2
REQ-TX-006	A payment exceeding the sender's balance is rejected with an error	P1

2.5 Notifications (REQ-NOTIF)

ID	Requirement	Priority
REQ-NOTIF-001	A user receives a notification when a payment is received or a request is created for them	P1
REQ-NOTIF-002	A user can dismiss notifications; the unread counter updates	P2

3. Requirements Traceability Matrix

The RTM proves every requirement has at least one test case and shows whether coverage is automated. Test case details live in Document 2 (Test Case Definitions).

Coverage key: **A** = automated (passing spec in CI) · **D** = documented (issue written, not automated) · **P** = planned (cataloged) · — = no coverage.

Requirement	Test cases	Coverage	Gap / next action
REQ-AUTH-001	TC-001 (retired), TC-005, TC-022, Vitest auth suite	A	Automate TC-005 collection
REQ-AUTH-002	TC-004, TC-005, TC-021, Vitest auth suite	A	Finish TC-004 assertions (stays on /signin, no cookie stored) UI
REQ-AUTH-003	TC-006	P	Write Cypress spec
REQ-AUTH-004	TC-007	P	Write Cypress spec
REQ-AUTH-005	TC-008, Vitest auth suite (cookie reuse)	A (API level)	UI-level check in TC-008

REQ-USER-001	TC-017	P	Write Vitest spec for <code>/users/search</code>
REQ-USER-002	—	—	Add test case (P2)
REQ-USER-003	TC-018	P	Write Playwright spec
REQ-BANK-001	TC-009	P	Write Cypress spec
REQ-BANK-002	TC-010	P	Add to Postman collection
REQ-BANK-003	—	—	Add test case (P2)
REQ-TX-001	TC-002, TC-019	D	Automate TC-002 in Cypress
REQ-TX-002	TC-003	D	Automate TC-003 in Cypress
REQ-TX-003	TC-011	P	Write Cypress spec
REQ-TX-004	TC-012, TC-013	P	Write Cypress specs
REQ-TX-005	TC-015	P	Write Cypress spec
REQ-TX-006	TC-014	P	Write Vitest spec
REQ-NOTIF-001	TC-016	P	Write Cypress spec
REQ-NOTIF-002	—	—	Add test case (P2)

Coverage summary (2026-08-02): 19 requirements — 3 with automated coverage, 2 documented, 11 planned, 3 with no test case yet (all P2). TC-021 and TC-022 were raised retroactively to trace auth UI tests that were already coded in `web/cypress/e2e/rwa/auth.cy.ts`.

4. Maintenance Rules

1. The RTM is updated in the same pull request that adds, automates, or retires a test case — never in a separate “docs later” commit.
2. No requirement moves to “Done” with a — row in the matrix.
3. All P0/P1 rows must reach **A** before the project is considered complete (exit criterion).
4. Requirement IDs are immutable; a changed requirement gets a new row and the old one is marked superseded, preserving history.